

Pokédex Search

A Coveo FDE technical challenge

Franck Benichou · Forward Deployed Engineer candidate

 Live app · pokemon-search-one-chi.vercel.app

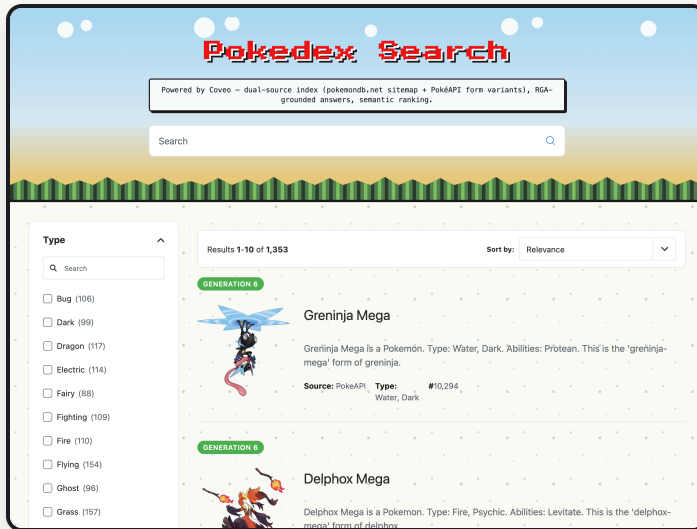
 GitHub · github.com/benichou/coveo-pokemon-challenge

 RGA performance monitoring · pokemon-rga-dashboard.vercel.app

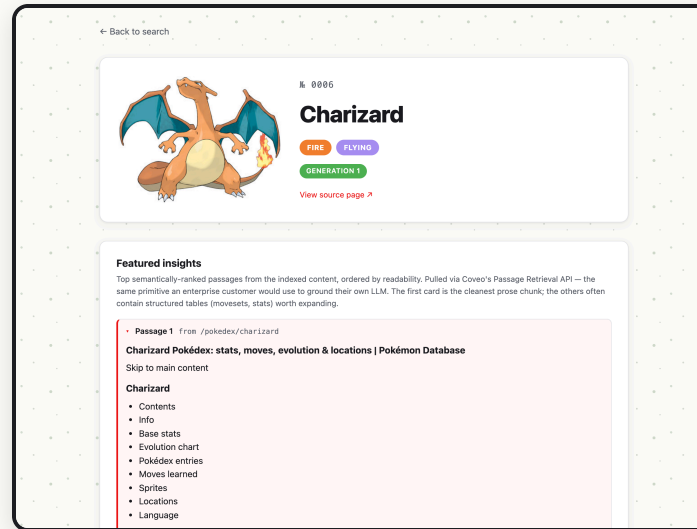
 Query observability · [grafana public dashboard](https://grafana-public-dashboard)

What I built

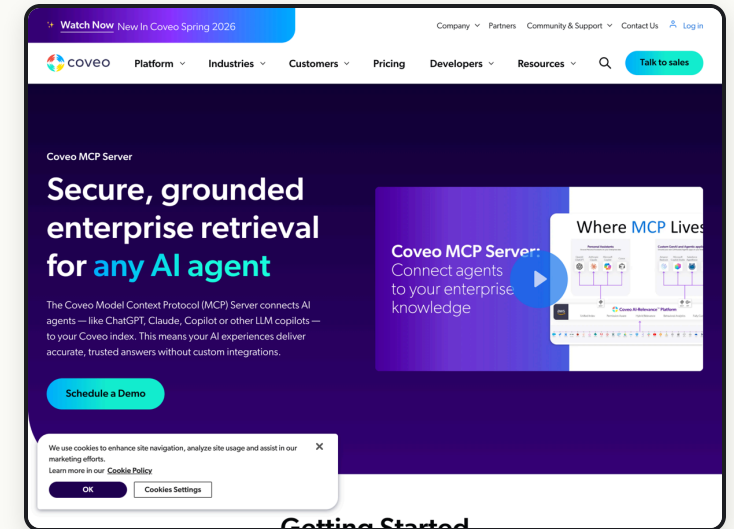
A Pokémon-themed search experience on Coveo Cloud —
three user-facing surfaces powered by **one Coveo brain**.



Atomic main page

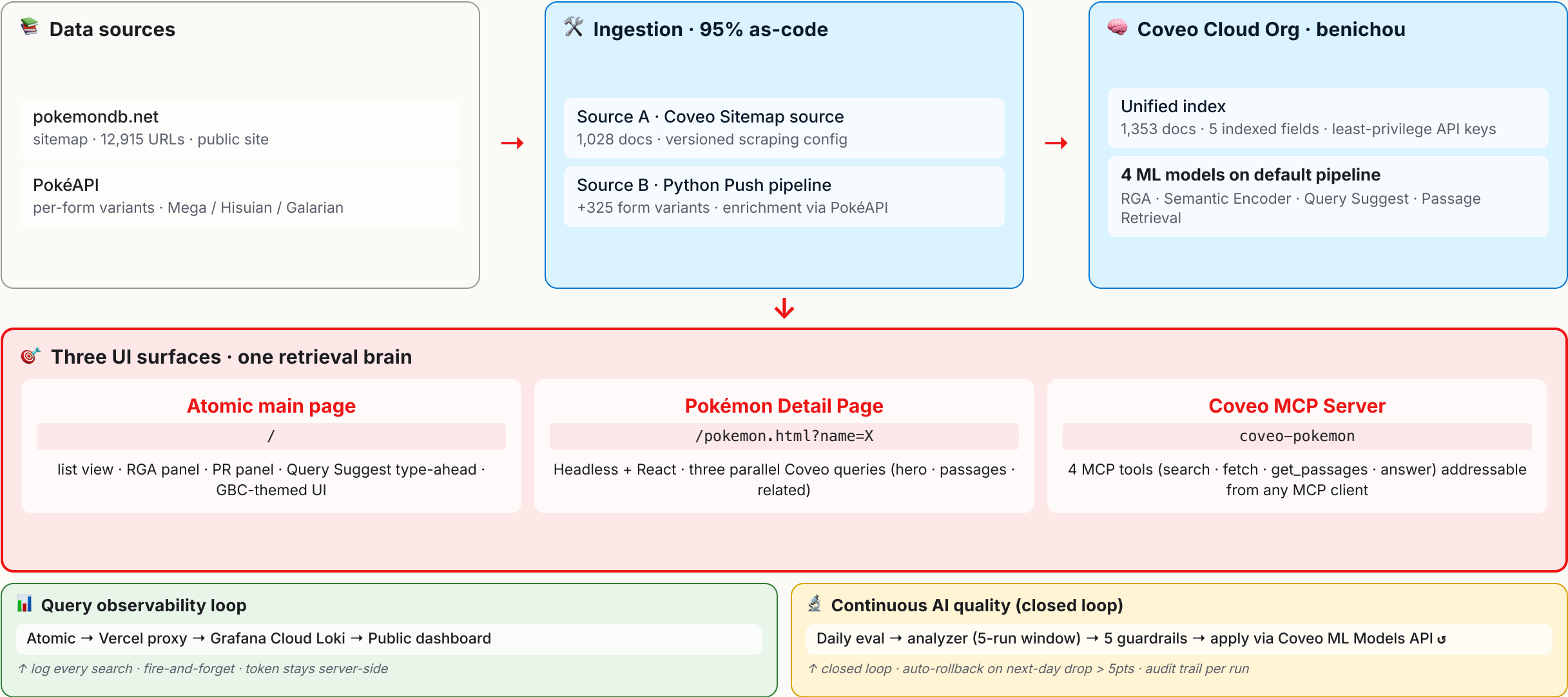


Pokémon Detail Page



Coveo MCP Server

Architecture · one Coveo org, three UI surfaces, two loops



Live demo · switching to the browser

Pokedex Search

Powered by Coveo – dual-source index (pokemondb.net sitemap + PokéAPI form variants), RGA-grounded answers, semantic ranking.

🔍 Atomic main page · pokemon-search-one-chi.vercel.app

← Back to search

📄 Pokémon Detail Page (Headless + React)

🌟 Watch Now New In Coveo Spring 2026

Company ▾ Partners Community & Support ▾ Contact Us [Log in](#)



Platform ▾

Industries ▾

Customers ▾

Pricing

Developers ▾

Resources ▾



Talk to sales

🤖 Coveo MCP Server (coveo-pokemon)

RGA Skill Evaluator

Daily quality scorecard for Coveo's RGA on the Pokemon index

[GitHub](#) ➔

Latest run — 2026-06-04

n = 100 questions · judge = c:laude-sonnet-4-5-20250929 · Δ vs 2026-06-03

📊 RGA performance monitoring

Coveo Pokemon Search — query observability

⏪ ⌚ Last 24 hours ▾ ⏩ 🔍 Refresh 30s ▾

Top queries (selected window) ⓘ

pikachu

1

RGA fire rate (selected window) ⓘ

16.7%

📊 Query observability

Follow along — full source at github.com/benichou/coveo-pokemon-challenge

github.com/benichou/coveo-pokemon-challenge

Why dual-source ingestion

The **1,353 docs mixing in the result list** during the demo — that's two ingestion sources unified into one Coveo index.

	Source A · Coveo Sitemap source	Source B · Python Push pipeline
Effort	Zero code · just a versioned scraping config	Python pipeline (<code>push-pokemon/</code>)
Throttling	Coveo handles automatically	I handle (PokéAPI 100 req/min cap)
Freshness	Refresh on a schedule	On-demand re-push
Form variants	One doc per canonical slug	One doc per form (Mega · Hisuian · Galarian)
Use for	Canonical Pokémon pages	Per-form coverage + PokéAPI enrichment

Not *"one source is right"* — different sources serve different facets of the same data.

Coveo config — code vs Console

Everything in the demo's search behavior is reproducible from code. Coveo gates a small set of one-time setup steps to the Console.



Code-as-source-of-truth

```
config/
├── fields.json          · Coveo resource declarations
├── source/              · 5 indexed fields
├── ml/                  · Sitemap A: definition · scraping · URL filter
└── mcp/pokemon-mcp.yaml · QS seed CSV (152 weighted queries)
                        · MCP Server config (manually mirrored)

scripts/
├── bootstrap.sh         · bash · Python · idempotent · REST apply
├── validate/            · one-command full provisioning
├── setup/               · API keys + org features preflight
├── source/              · fields · source · scraping · mappings
├── ml/                  · widen filter · rebuild · poll
├── mcp/discover_api.sh  · associate 4 models · seed Query Suggest
└── audit/               · probes for admin REST API (404 confirmed)
                        · PokéAPI cross-ref · index leak detection

push-pokemon/           · Python pipeline · Source B implementation
                        · scrape pokemondb · enrich via PokéAPI ·
                        · transform · POST to /push/v1/.../documents

rga-eval/
rga-closed-loop/
├── prompts/pokemon-rga.yaml · 100-Q golden + Sonnet 4.6 judge · drives loop
├── prompts/history/        · daily prompt tuning
├── src/apply.py             · RGA Custom Prompt (PUT to ML Models API)
├── src/closed_loop_run.py   · every prior version, dated YAMLS
└── src/guardrails.py        · dry-run default · --apply to write
                        · cron orchestrator
                        · confidence · lift · rate-limit · sanity

.github/workflows/
├── rga-eval-daily.yml       · autonomous cron-driven Coveo updates
└── closed-loop-daily.yml    · 06:00 UTC · eval + bot-commit
                        · 06:30 UTC · analyzer + apply via API
```



Console-only (Coveo product design)

- **Org creation** — Coveo vendor onboarding flow; can't bootstrap via API
- **6 API keys** — Coveo shows secrets once at creation, by design (one-time per key)
- **ML model creation** — RGA · SE · QS · PR. Console-only per Coveo docs. Only the *post-creation behavior* (prompts, seed queries, pipeline associations, builds) is API-scriptable.
- **MCP Server creation + admin** — no public admin REST API yet (verified via `scripts/mcp/discover_api.sh` — 8 candidate endpoints all 404). `config/mcp/pokemon-mcp.yaml` is the source-of-truth; the Console is currently a manual mirror.

Four ML models · one pipeline

All four models attached to one query pipeline — the challenge brief asks for RGA and Query Suggest; I added the other two because the index already supports them.

▸ RGA

Relevance Generative Answering

LLM-grounded answer + citations above the result list.

Custom Prompt versioned in `rga-closed-loop/prompts/pokemon-rga.yaml` · **v1.1.0 live** · 8 rules

↳ The prompt itself is autonomously tunable — see closed-loop slide

▸ SE

Semantic Encoder

Embedding-based relevance — query/doc similarity beyond keyword overlap.

Console-created · associated via `scripts/ml/associate_models.sh` · no prompt config needed

↳ Recovers "what type is charizard" even when the doc says "Fire/Flying"

▸ QS

Query Suggest (type-ahead)

In-search-box suggestions as the user types.

152 weighted seed queries in `config/ml/default-queries.json` · CSV PUT to Coveo's Default Queries endpoint

↳ Solved cold-start without UA event synthesis · see `docs/ml-models.md`

▸ PR

Passage Retrieval (bonus)

Returns ranked text passages — chunked grounding primitive.

Console-created · associated to default pipeline · powers two UI surfaces (Atomic + Detail Page)

↳ Same primitive an enterprise would use for CRGA — bridge to Topic 2

Three UI surfaces + cross-cutting ops skills

Same Coveo brain · different consumption modes. **The third surface (MCP) is for AI agents — that's the 2026 story.**

`/pokemon-mcp` is its 1:1 driver (in the MCP card). The other two skills sit across all three surfaces.

BROWSER USERS

Atomic main page

List view · facets · RGA panel · Passage Retrieval
· Query Suggest type-ahead · GBC-themed UI.

Vite + `@coveo/atomic` · 5 rotating biome themes ·
Press Start 2P pixel font

Drive it: open `pokemon-search-one-chi.vercel.app`

BROWSER USERS

Pokémon Detail Page

Hero · Featured insights (PR) · Related grid —
three parallel Coveo round-trips on one page.

Vite + `@coveo/headless` + React · two Headless
engines · multi-entry build

Drive it: click any Atomic result, or
`/pokemon.html?name=X`

AI AGENTS

Coveo MCP Server

4 tools — `search` · `fetch` · `get_passages` ·
`answer` — addressable from any MCP client.

Coveo Hosted MCP · per-org endpoint · server
instructions = LLM system prompt

Drive it: `/pokemon-mcp` demo in Claude Code

↑ TWO CLAUDE CODE SKILLS OPERATE ACROSS ALL THREE SURFACES ABOVE — THEY'RE FDE OPS, NOT NEW SURFACES

`/rga-eval`

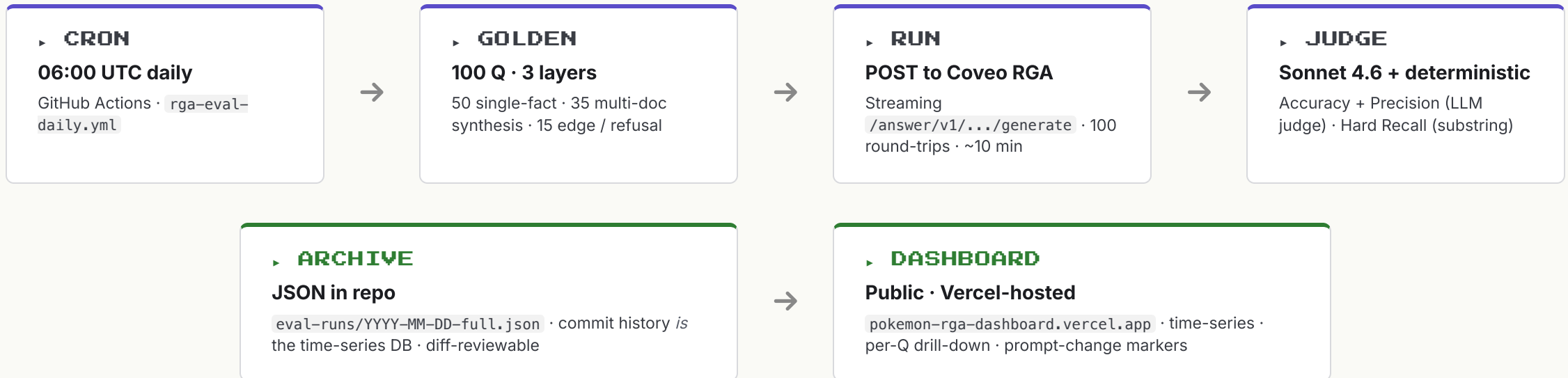
Measure RGA answer quality continuously — see next slide

`/rga-closed-loop`

Tune the RGA prompt autonomously — see slide after that

rga-eval · measure RGA quality every day

Bonus functionality. Most enterprise AI deployments ship, regress silently, retune in Slack threads. I built the alternative — a continuous quality scorecard that runs on its own.



↓ This JSON archive is the **input that drives the next slide** — the closed loop reads the 5-day eval window every morning at 06:30 UTC

▶ DRIVEN FROM CLAUDE CODE VIA

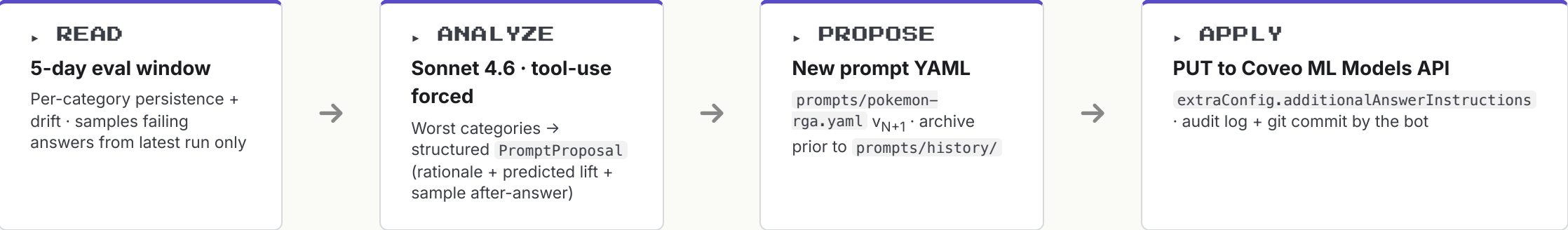
/rga-eval

Run a full 100-Q evaluation · show the latest results · drill into per-question outcomes

rga-closed-loop · the system tunes itself

Measurement without action is just a dashboard nobody reads. The closed loop **acts** on the eval signal — and only when 5 guardrails pass. **More bonus functionality.**

↑ Reads what the previous slide produces — yesterday's eval-run JSON · without that, this loop has nothing to act on



⌚ Tomorrow's 06:00 UTC eval measures the lift · the loop repeats

► FIVE GUARDRAILS BLOCK ANY UNSAFE APPLY — CONFIDENCE · LIFT · RATE-LIMIT · SANITY · AUTO-ROLLBACK

- Confidence ≥ 0.80
- Predicted lift $\geq +5\text{pts}$
- Rate-limit ≥ 3 days between applies
- Sanity (no empty / $2\times$ length)
- Auto-rollback if next-day drops $> 5\text{pts}$

► DRIVEN FROM CLAUDE CODE VIA

/rga-closed-loop

Run analyzer locally · dry-run apply · inspect audit log · rollback to previous version

What I learned

Three lessons that actually changed how I'd approach the next Coveo engagement.

▶ LESSON 1

Inspect the data *before* choosing the ingestion strategy.

Pokémon look uniform at first — name · type · dex number. They're not. pokemondb publishes **one canonical page per slug**; PokéAPI exposes **per-form endpoints** for Mega-X, Mega-Y, Hisuian, Galarian variants. The dual-source approach — Sitemap for canonical pages, Push for form variants — fell out of an hour looking at actual HTML, not from a design pattern.

↳ *Ingestion architecture is a data question, not a tooling question · source choice = data inspection × downstream use case.*

▶ LESSON 2

Closed loops compound. Static prompts decay.

The daily eval + auto-tuning machinery felt like over-engineering before it ran. Once the cron started firing every morning, the cognitive load of "is the prompt still good?" went to zero — the system answers that for me.

↳ *Build the measurement loop FIRST · you only know a prompt is "final" by measuring it over time.*

▶ LESSON 3

Vector + LLM doesn't model *relationships* — that's the 2026 frontier.

Some queries are relational — "*which Fire-types evolve from Water-types?*", "*which abilities counter Fire moves?*". SE embeds entities; RGA composes language — neither models the **edges between entities**. Coveo's stack today (Catalog = commerce-only · Knowledge Hub = content mgmt · no GraphRAG) leaves multi-hop reasoning to a future layer.

↳ *Multi-hop questions will need a knowledge-graph layer next · a natural 2026 extension of the RGA story.*

Production hardening · what I'd ship next

The build runs and behaves. **It's not production-grade yet.** Four honest gaps below — each one is a 2-week sprint, not a research project.

▸ HOSTING · SCALE · CONCURRENCY

Vercel → private cloud network

Today: Vercel auto-scales when requests arrive · Coveo handles their side · single region · first request after idle is slow ("cold start").

Gap: Never tested with simulated heavy traffic — simultaneous-user ceiling unknown · no fallback if our region goes down · no private network isolation.

↳ Next: simulated-traffic tests to find the limit · AWS managed functions + load balancer + CDN in a private network · multiple regions at once · explicit uptime + speed targets.

▸ MONITORING BEYOND AI QUALITY

2 dashboards → full app monitoring + targets

Today: AI-answer-quality dashboard + Grafana query logs · rest is Vercel defaults.

Gap: No app-performance tracing (where time goes in each request) · no error tracking that pings me when something breaks · no scheduled bots that fake-visit to detect downtime · **no documented uptime targets** · no automated paging.

↳ Next: app-performance monitoring (Datadog or Tempo+Grafana) · error tracking (Sentry) · scheduled uptime checks · written uptime + quality targets · automated paging when targets break.

▸ AUTH · DATA ACCESS · SECURITY

API keys → short-lived per-user tokens

Today: 6 narrowly-scoped API keys in `.env` / Vercel / GitHub · public site, no login · MCP uses a shared key.

Gap: Frontend uses **API key directly** instead of a per-user token · no login (Okta/Azure AD) · no **per-user document filtering** from source systems (Salesforce/SharePoint/Confluence) · keys in env files, not a vault · no rotation.

↳ Next: backend mints a fresh token per user · Okta/Azure AD single sign-on · **Coveo Security Identities** — pull each user's source permissions at index time, enforce at search · vault for keys · quarterly rotation.

▸ AUTONOMOUS LOOP · OPERATIONAL TRUST

Can we trust the loop to run unattended?

Today: 5 safety checks · automatic rollback if next-day quality drops · new prompts go live to *100% of users immediately* when checks pass.

Gap: Safety checks never triggered by a real bad prompt — we don't know they actually fire · daily eval scores 100 questions *we wrote*, doesn't measure if real users *click* the answers · no alert per apply (silent commit).

↳ Next: deliberately push a bad prompt to prove rollback works · roll out new prompts **10% → 50% → 100%** via Coveo's A/B framework (real user clicks decide the winner) · Slack/PagerDuty alert per apply.

▸ MORE COVEO PLATFORM DEPTH I'D ACTIVATE AT SCALE

ART · clicks-trained relevance

DNE · ML-tuned facets

IPE · index-time Python

UA Data Service · native analytics export

Catalog · entity hierarchies

Thank you Q&A

Three UI surfaces · two loops · one Coveo brain

Built as code · measured continuously · ready to tune itself overnight

 Live app · pokemon-search-one-chi.vercel.app

 GitHub · github.com/benichou/coveo-pokemon-challenge

 RGA performance monitoring · pokemon-rga-dashboard.vercel.app

 Query observability · [grafana public dashboard](https://grafana-public-dashboard)

Appendix · References

Every Coveo claim in the deck links to its source · external references are listed for landscape context.

▸ COVEO DOCUMENTATION

SECURITY & DATA ACCESS (SLIDE 12)

- [Management of security identities & item permissions](#)
- [Security identity cache and provider](#)
- [Content security options](#)

A/B TESTING & ML ROLLOUT (SLIDE 12)

- [Manage A/B tests](#)
- [Manage model associations with query pipelines](#)

ML MODELS (SLIDES 7, 12)

- [About Automatic Relevance Tuning \(ART\)](#)
- [Associate a DNE model with a pipeline](#)
- [Associate a QS model with a pipeline](#)
- [Inspect Coveo ML models](#)

KNOWLEDGE GRAPH / GRAPHRAG STATUS (SLIDE 11)

- [Coveo Knowledge Hub](#)
- [Catalog entity \(commerce-only\)](#)
- [Query extension for related entities](#)

▸ EXTERNAL REFERENCES

GRAPHRAG 2026 LANDSCAPE (SLIDE 11)

- [GraphRAG in 2026 — Practical Buyer's Guide](#)
- [Microsoft GraphRAG \(research pipeline\)](#)
- [WRITER Knowledge Graph \(writer.com/product/graph-based-rag\)](#)
- [Neo4j Agentic GraphRAG](#)

DATA SOURCES (SLIDE 5)

- [pokemondb.net sitemap](#) (12,915 URLs)
- [PokéAPI](#) (per-form endpoints)

INFRASTRUCTURE & RUNTIME

- [Vercel](#) · hosting + serverless proxy
- [Grafana Cloud](#) · free-tier Loki for query logs
- [GitHub Actions](#) · daily eval + closed-loop crons
- [Anthropic Claude API](#) · Sonnet 4.6 (judge + analyzer)

TOOLING

- [Marp](#) (deck) · [Mermaid](#) (diagrams) · [uv](#) (Python) · [Vite](#) (frontend)

Appendix · Repo map

Key files to open during/after Q&A — each anchors a specific claim from the deck.

▸ rga-eval · continuous AI quality (slide 9)

`rga-eval/golden/questions.json` · 100-Q dataset (50 / 35 / 15 layers)
`rga-eval/src/main.py` · daily eval runner (~10 min)
`rga-eval/src/llm_judge.py` · Sonnet 4.6 — Accuracy + Precision
`rga-eval/src/metrics.py` · deterministic Hard Recall
`eval-runs/*.json` · commit history = time-series DB

▸ rga-closed-loop · autonomous tuning (slide 10)

`rga-closed-loop/prompts/pokemon-rga.yaml` · live RGA prompt (v1.1.0)
`rga-closed-loop/prompts/history/*.yaml` · every prior version archived
`rga-closed-loop/src/analyzer.py` · Sonnet 4.6 PromptProposal
`rga-closed-loop/src/guardrails.py` · 5 safety checks
`rga-closed-loop/src/closed_loop_run.py` · cron orchestrator

▸ atomic-search · three UI surfaces (slide 8)

`atomic-search/src/main.js` · Atomic main page entry
`atomic-search/src/pokemon-detail/App.tsx` · Headless + React detail
`atomic-search/src/observability.js` · per-query Grafana logger
`atomic-search/api/log-query.js` · Vercel serverless proxy

▸ MCP · agent-facing surface (slide 8)

`config/mcp/pokemon-mcp.yaml` · MCP server source-of-truth
`.claude/mcp.json` · Claude Code HTTP MCP wiring
`.claude/skills/pokemon-mcp/SKILL.md` · /pokemon-mcp driver
`scripts/mcp/discover_api.sh` · admin REST API probe

▸ docs · panel-shareable methodology

`docs/rga-eval-methodology.md` · how we measure AI quality
`docs/observability.md` · what we log + why
`docs/caching-strategy.md` · cache trade-offs
`docs/mcp-integration.md` · MCP setup + workflow
`docs/detail-page.md` · Headless + React architecture

▸ infrastructure-as-code (slide 6)

`config/` · Coveo resource declarations (JSON / YAML)
`scripts/bootstrap.sh` · one-command full provisioning
`.github/workflows/rga-eval-daily.yml` · 06:00 UTC eval cron
`.github/workflows/closed-loop-daily.yml` · 06:30 UTC auto-tune
`observability/grafana-dashboard.json` · dashboard-as-code